Projektsteckbrief: Seeland Mobil

Carsharing-Verwaltung mit Spring Boot und PostgreSQL

Database Development, Projektarbeit (Einzelarbeit) · Mohamed Rumy · Dozent: Simon Erhardt · Stand 10.10.2026

1 Thema

Ausgangslage: Ein kleiner Carsharing-Anbieter im Seeland hat Fahrzeuge an mehreren Standorten (z. B. Biel, Lyss, Nidau). Die Reservationen werden heute in einer Excel-Liste geführt. Dabei kommt es immer wieder zu Doppelbuchungen, und niemand weiss genau, welche Fahrzeuge und Standorte wie stark genutzt werden.

Nutzen: Eine REST-API verwaltet Fahrzeuge, Kunden und Reservationen zentral. Die Datenbank verhindert Überschneidungen auch dann, wenn zwei Personen gleichzeitig buchen. Beim Abschluss einer Fahrt wird der Kilometerstand nachgeführt, und Auswertungen zur Nutzung helfen bei der Planung der Flotte.

Abgrenzung: Kein Login, keine Fahrzeugortung, keine Tarife und keine Zahlung. Die Abholung wird nicht separat erfasst; eine Fahrt beginnt mit dem Reservationsbeginn. Demo über Swagger UI und eine einfache Oberfläche (freiwillig).

2 Fachliche Anforderungen

Verletzt eine Anfrage eine Geschäftsregel, antwortet die API mit 409 Conflict, bei ungültigen Eingaben mit 400. Das Ende eines Zeitraums gehört nicht dazu: Reservationen bis 14 Uhr und ab 14 Uhr überschneiden sich nicht.

A1 Fahrzeuge verwalten

Fahrzeuge mit eindeutigem Kennzeichen, Modell, Kategorie (Klein, Kombi, Transporter) und Standort erfassen, ändern und ausser Betrieb nehmen; Standortwechsel und Ausserbetriebnahme nur ohne offene Reservationen. Danach ist das Fahrzeug weder reservierbar noch in der Suche (A6). Änderungen mit veraltetem Stand werden abgelehnt.

> *Prüfung:* Doppeltes Kennzeichen, veraltete Version, Standortwechsel oder Ausserbetriebnahme bei offener Reservation: abgelehnt.

A2 Kunden verwalten

Kunden mit Name und E-Mail erfassen, ändern und sperren. Die E-Mail ist eindeutig, ohne Rücksicht auf Gross-/Kleinschreibung. Gesperrte Kunden können nicht mehr reservieren.

> *Prüfung:* Doppelte E-Mail («anna@…» / «Anna@…») und Reservation eines gesperrten Kunden werden abgelehnt.

A3 Reservieren und stornieren

Ein Kunde reserviert ein Fahrzeug für einen Zeitraum. Das Ende liegt nach dem Beginn, der Beginn nicht in der Vergangenheit. Zwei nicht stornierte Reservationen desselben Fahrzeugs dürfen sich nie überschneiden – auch nicht bei gleichzeitigen Anfragen. Eine Reservation kann bis zum Beginn storniert werden; sie bleibt mit Status STORNIERT gespeichert und gibt den Zeitraum wieder frei.

> *Prüfung:* Ist 10–14 Uhr reserviert, wird 12–16 Uhr abgelehnt, 14–16 Uhr aber angenommen. Nach dem Storno von 10–14 Uhr ist 12–16 Uhr möglich. Ein Storno nach Beginn wird abgelehnt, ein Ende vor oder gleich dem Beginn ergibt 400.

A4 Reservationen eines Kunden anzeigen

Die Reservationen eines Kunden werden seitenweise angezeigt (20 pro Seite, höchstens 100), die neueste zuerst (bei gleichem Beginn nach ID), optional nach Status gefiltert, jeweils mit Kennzeichen und Abholstandort.

> *Prüfung:* Beim Blättern über mehrere Seiten fehlt keine Reservation und keine kommt doppelt vor; Filter, Sortierung und Seitenbegrenzung laufen in der Datenbank.

A5 Fahrt abschliessen

Bei der Rückgabe werden Rückgabezeit und Kilometerstand erfasst: Fahrt anlegen, Reservation auf ABGESCHLOSSEN setzen, Kilometerstand des Fahrzeugs aktualisieren – alles oder nichts. Der neue Kilometerstand darf nicht kleiner sein als der bisherige, die Rückgabezeit liegt zwischen Beginn und jetzt.

> *Prüfung:* Alle drei Änderungen sind sichtbar; bei zu kleinem Kilometerstand bleibt alles unverändert (Kapitel 4). Ein zweiter Abschluss wird abgelehnt.

A6 Verfügbare Fahrzeuge suchen

Für einen Zeitraum werden alle freien Fahrzeuge in Betrieb angezeigt, optional nach Standort und Kategorie gefiltert, sortiert nach Kennzeichen und seitenweise.

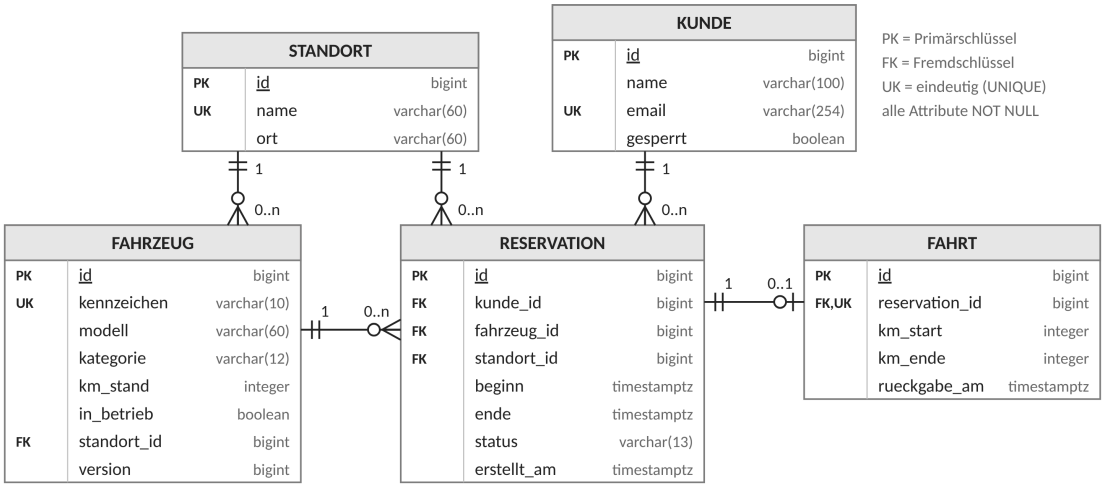
> *Prüfung:* Bei einer Reservation 10–14 Uhr fehlt das Fahrzeug in der Suche 13–15 Uhr, bei 14–16 Uhr erscheint es.

A7 Nutzung auswerten

Für einen Zeitraum pro Fahrzeug und pro Standort: Anzahl abgeschlossene Fahrten, Nutzungsstunden (Rückgabe minus Beginn), gefahrene Kilometer und Auslastung (Nutzungsstunden geteilt durch Anzahl Fahrzeuge × Stunden im Zeitraum; je Standort zählen die ihm heute zugeordneten Fahrzeuge in Betrieb). Massgebend sind der Reservationsbeginn und der Abholstandort der Reservation; stornierte und offene Reservationen zählen nicht, Standorte und Fahrzeuge ohne Fahrt erscheinen mit 0.

> *Prüfung:* Testdaten mit bekannten Summen (inkl. Fahrzeug ohne Fahrt) liefern genau diese Werte, nach einem Fahrtabschluss sofort den neuen.

3 Modellskizze



- «Keine Überschneidung» setze ich in PostgreSQL mit einem Exclusion-Constraint um (gleiches Fahrzeug, überlappender Zeitraum, nur nicht stornierte Reservationen). Er greift auch bei gleichzeitigen Zugriffen.
- FAHRT ist eine eigene Tabelle, weil Rückgabedaten erst beim Abschluss entstehen (keine leeren Spalten).
- RESERVATION hält mit `standort_id` den Abholstandort fest (nachfolgende Migration). Wird ein Fahrzeug verschoben, bleiben frühere Fahrten beim Standort, an dem sie begonnen haben.
- `km_stand` ist bewusst redundant (aus der letzten Fahrt berechenbar) und wird in A5 mitgeführt.
- Kategorie, Status, «Ende nach Beginn» und «km_ende ≥ km_start» sind CHECK-Constraints.

4 Kritische Geschäftsoperation

Atomarität: Fahrt abschliessen (A5)

Drei Schreibzugriffe gehören zusammen und laufen in einer Transaktion:

1. Reservation von AKTIV auf ABGESCHLOSSEN setzen (nur wenn sie noch AKTIV ist).
2. Fahrt einfügen; `km_start` ist der bisherige Kilometerstand des Fahrzeugs.
3. Kilometerstand des Fahrzeugs aktualisieren.

Für den Rollback-Test gebe ich einen zu kleinen Kilometerstand an. Schritt 1 ist dann schon geschrieben, Schritt 2 scheitert am CHECK-Constraint. Danach darf nichts geändert sein: Reservation weiterhin AKTIV, keine Fahrt, Kilometerstand unverändert.

Konfliktfall: gleichzeitige Reservation (A3)

Zwei Kunden reservieren gleichzeitig dasselbe Fahrzeug für überlappende Zeiten. Beide sehen vorher «frei»; ohne Schutz in der Datenbank entstehen zwei Reservationen. Mit dem Exclusion-Constraint wartet die zweite Transaktion, bis die erste fertig ist, und schlägt dann fehl; die API meldet 409 Conflict. Im Test halte ich die erste Transaktion vor dem Commit an, damit die Zugriffe wirklich gleichzeitig laufen. Zusätzlich hat FAHRZEUG eine Versionsnummer (optimistisches Locking), damit sich Änderungen am Fahrzeug und ein Fahrtabschluss nicht unbemerkt überschreiben.

5 Vorgesehene Auswertungen

- Welche Fahrzeuge sind in einem bestimmten Zeitraum an einem Standort frei? (A6)
- Welche Reservationen hat ein bestimmter Kunde? (A4)
- Wie viele Fahrten, Stunden und Kilometer hatte jedes Fahrzeug in einem Monat? (A7)
- Welcher Standort ist am stärksten ausgelastet? (A7)

Die Suche (A6) ist eine JPQL-Abfrage mit DTO-Projektion, die Nutzungsauswertung (A7) eine parametrisierte JDBC-Abfrage auf einer View der abgeschlossenen Fahrten. Die Standortauswertung wird gecacht und nach jedem Fahrtabschluss und jeder Änderung des Fahrzeugbestands eines Standorts geleert. RESERVATION fülle ich mit über 100'000 generierten Datensätzen. Für Index und N+1-Analyse eignet sich die Reservationsliste eines Kunden (A4).

Änderungen gegenüber der eingereichten Fassung vom 07.10.2026

- A1: Standortwechsel nur ohne offene Reservationen; Prüfung um Standortwechsel und veraltete Version ergänzt.
- A3, A4, A5: Prüfungen ergänzt (Ende ≤ Beginn, Filter in der Datenbank, zweiter Abschluss); A4 höchstens 100 pro Seite.
- A7 und Modellskizze: Zuordnung nach Abholstandort (`reservation.standort_id`), Fahrzeuge für die Standort-Auslastung präzisiert; Cache auch nach Änderungen des Fahrzeugbestands geleert.
- Abgrenzung: Oberfläche im Browser als freiwillige Ergänzung.